

Meta-características específicas do domínio

Johnattan D. F. Viana (jdfv@cin.ufpe.br)

Repositório: <https://github.com/johnattandouglas/DisciplinaMtL>

1. Introdução

O problema abordado é a seleção de algoritmo e de estratégia de balanceamento para Regressão Desbalanceada (IR). Utilizamos o meta-dataset proposto por Avelino et al. (Avelino et al. 2025), composto por 215 conjuntos de dados de regressão, provenientes do OpenML (Vanschoren et al. 2013) e Moniz et al. 2017. Cada conjunto de dados está associado a resultados de desempenho para diferentes algoritmos de aprendizagem e estratégias de reamostragem, que definem os meta-alvos.

O desempenho de cada combinação **algoritmo** $\times$ **estratégia de balanceamento** foi medido por: (i) **SERA**: área sob a curva de erro quadrático acumulado, integrada sobre limiares crescentes de relevância $\varphi(y)$; e (ii) **F1**: calculado sobre a binarização dos casos relevantes (via $\varphi(y) \geq \text{limiar}$). Os algoritmos candidatos são {BG, DT, MLP, RF, SVM, XG} e as estratégias de balanceamento são {NONE, GN, SMT, RU, RO, WC, SG}. O meta-alvo categórico (`model`, `strategy`) é o vencedor dessa grade $6 \times 7 = 42$ combinações, e o objetivo do meta-aprendizado é prever essa combinação vencedora a partir de meta-características do dataset, sem precisar rodar as 42 combinações em um dataset novo.

Para avaliar as meta-características potencialmente relevantes para este problema, foram consideradas e avaliadas meta-características extraídas (Seção 2) e representações aprendidas (Seção 3). Como será discutido na Seção de Resultados (Seção 4), uma parcela significativa das famílias clássicas de meta-características foi originalmente desenvolvida no contexto de problemas de classificação balanceada, o que pode limitar sua capacidade de caracterizar adequadamente datasets de IR. Em contraste, meta-características aprendidas podem capturar relações mais relevantes entre as propriedades dos datasets e os respectivos meta-targets.

2. Meta-features Extraídas

2.1. PyMFE

A extração foi feita usando os grupos padrão da biblioteca. Das 111 colunas originais, 56 foram descartadas por conterem NaN/Inf em pelo menos um dataset. As 56 colunas removidas são dos grupos *landmarking* e *model-based*, mais as medidas *statistical* baseadas em análise discriminante. Todas essas medidas pressupõem um alvo categórico com classes bem povoadas. Como o alvo aqui é contínuo e desbalanceado, o PyMFE não consegue computá-las de forma estável, e por isso retornam NaN/Inf. Esse foi o primeiro indício de que as famílias clássicas de classificação simplesmente não se aplicam bem a este problema.

Na família *general*, `nr\_inst/nr\_attr/attr\_to\_inst` são baratas e úteis para controle de escala, mas não capturam a distribuição contínua do alvo. Em custo computacional, a maioria é $O(n \cdot m)$, portanto o tempo é irrisório frente a outras famílias.

Na família *Statistical*, as medidas descrevem bem a geometria/dispersão de X e ajudam a antecipar o desempenho de modelos sensíveis à escala/covariância (p.ex., SVM vs. árvores). Essa família não examina a cauda do alvo ponderada por relevância. O custo inclui operações $O(m^2)$ (covariância, `nr\_cor\_attr`) e testes estatísticos (Shapiro-Wilk, ANOVA/discriminante) por atributo.

Na família *Information-theoretic*, as medidas exigem discretizar o alvo contínuo para calcular entropia; esse critério de discretização não se alinha a $\varphi(y)$, que define o que é “raro”. Assim, dois alvos podem ter a mesma entropia de classe e desbalanceamentos sob $\varphi(y)$ muito distintos, e a discretização genérica não localiza os extremos relevantes para SERA/F1. Ainda assim,

mut_inf/ns_ratio são úteis para indicar o quão previsível é o alvo a partir de X , independentemente do desbalanceamento. A implementação requer discretização (baixo custo) e cálculo de entropias ($O(n)$ por atributo).

2.2. ECoL

Esta biblioteca, implementada em R, cobre a família de complexidade de dados, que é importante por descrever a dificuldade geométrica do problema de aprendizado em si, e não apenas propriedades dos atributos. Ao contrário das medidas *landmarking/model-based* do PyMFE (que exigiram um alvo categórico e por isso falharam), as medidas da ECoL operam diretamente sobre o alvo contínuo. Portanto, esta família é a que mais faz sentido ao fato de o problema ser regressão. Em particular, **S1/S2** (baseadas em MST) são sensíveis a mudanças bruscas locais do alvo, e **C3/C4** (eficiência de atributo) são explicitamente medidas de quantos exemplos atrapalham. Ambas podem ser úteis para explicar o fenômeno de raridade que o problema precisa capturar, mesmo sem usar $\varphi(y)$ explicitamente. É por isso que a ECoL entra como segunda fonte de meta-características extraídas no lugar de, por exemplo, medidas clássicas de complexidade para *classificação*.

No entanto, nenhuma dessas medidas pondera explicitamente $\varphi(y)$, e todas tratam cada instância com peso igual. Esse é o principal motivo, para também investigar representações aprendidas, que podem conseguir aprender pesos ponderados aos atributos pouco representados. Quanto ao custo, cada extração ajusta um modelo auxiliar (regressão linear, PCA, MST, 1-NN) por dataset, e por essa razão tende a ser mais caro de computar do que PyMFE.

3. Meta-features Aprendidas

3.1. Dataset2Vec

Essa abordagem aplica um encoder invariante a permutação que é treinado, sobre uma grande coleção de datasets tabulares diversos, para mapear amostras de submatrizes a um vetor de dimensão fixa $d = 32$. Nestes experimentos, as 32 meta-features foram extraídas com os pesos do modelo que foram disponibilizados pelos autores na versão treinada. Essa abordagem pode fazer sentido para o problema de IR porque, em vez de decidir manualmente qual fórmula sinaliza dificuldade sob desbalanceamento, delega-se essa decisão à rede. Essa metodologia pode ser proveitosa porque nenhuma família isolada captura bem a raridade de extremos em y .

No entanto, o Dataset2Vec original foi pré-treinado com um objetivo de similaridade entre datasets tabulares genéricos, sem qualquer rótulo relacionado a raridade de valores em IR. Ou seja, nada na função objetivo usada para o treinamento da rede incentiva a aprendizagem a capturar os aspectos do IR. Isso é evidente porque nos experimentos, o D2V não teve bom desempenho. Isso evidencia que este embedding carrega pouca informação específica do problema de IR.

Quanto ao custo computacional, a extração é barata porque o modelo já está treinado, mas depende de um custo de pré-treino. A extração foi feita utilizando a versão do modelo disponibilizada pelos autores.

3.2. TabPFN

Esse *foundation model* extrai representações que também estão fora da taxonomia clássica. TabPFN é um transformer pré-treinado sobre milhões de datasets tabulares sintéticos, capaz de fazer inferência bayesiana sem re-treino por tarefa. Esse modelo recebe o conjunto de treino (X, y) inteiro como tokens de entrada e produz previsões via atenção entre instâncias.

Neste relatório, o modelo foi usado de modo a salvar a representação latente interna (saída intermediária do transformer) de cada instância de treino, com dimensão fixa 512. Essas representações por instância foram então agregadas por média simples ao longo de todas as

instâncias, produzindo um único vetor de 512 posições por dataset. Na versão padrão, a representação é limitada até 1000 linhas amostradas por dataset. Essa definição foi mantida.

Por utilizar mecanismos de atenção, é provável que interações não-lineares entre atributos e a forma como pontos individuais (inclusive extremos) influenciem a representação aprendida. Os resultados indicam esta vantagem. O TabPFN teve bom desempenho, indício de que o embedding aprendido carrega sinal relevante para decidir a estratégia de balanceamento que nenhuma família clássica isolada captura tão bem.

Por outro lado, o custo computacional do TabPFN é elevado em comparação com PyMFE e ECoL. O carregamento do modelo pré-treinado levou 21,71 segundos e acrescentou aproximadamente 499 MB ao consumo de RAM. O modelo possui 58.274.944 parâmetros, correspondendo a aproximadamente 222 MB de pesos. Após o carregamento, a extração das representações levou, em média, 4,09 s por dataset, totalizando 900,6 s para os 220 datasets avaliados em CPU. Esse tempo corresponde a aproximadamente 7 vezes o custo de extração por dataset observado para o PyMFE.

4. Resultados

Em vez de concatenar todas as fontes de meta-características em uma única matriz $X_{\text{estendido}}$ (o que levaria a $55 + 41 + 512 + 32 = 640$ colunas), optou-se por uma comparação tipo ablação, na qual cada fonte de meta-características foi avaliada isoladamente.

4.1. Desempenho em nível meta e nível base

A Tabela 4.2 mostra, por meta-alvo, o desempenho de cada representação. Destaca-se que o *fine-tuning* da nossa representação proposta (Our-Full), orientado ao meta-alvo, captura sinal que nenhuma família clássica isolada captura.

Representação	model		strategy	
	acc	F1-macro	acc	F1-macro
Our-Full	0,679	0,612	0,358	0,244
TabPFN	0,479	0,379	0,451	0,191
ECoL	0,460	0,361	0,386	0,154
PyMFE	0,433	0,354	0,335	0,177
D2V	0,377	0,310	0,288	0,158
Our-Init	0,326	0,269	0,298	0,168
Random	0,288	0,148	0,326	0,099

Tabela 1. Acurácia e F1-macro na predição dos meta-alvos (model e strategy).

Em nível base, a média de SERA por representação varia entre $2,6 \times 10^{10}$ e $6,4 \times 10^{10}$ — cerca de 10 ordens de grandeza acima da mediana (2–8), porque um pequeno número de datasets tem erro catastrófico sob algumas combinações model-strategy. Isso é coerente com o próprio desenho do SERA que faz a penalização quadrática crescente sobre os exemplos raros.

Para F1, o teste de Friedman rejeitou a hipótese de igualdade entre as representações ($\chi^2 = 89,50$, $p = 3,8 \times 10^{-17}$). A diferença crítica de Nemenyi foi $CD = 0,614$, com os seguintes rankings médios: Our-Full (3,44), TabPFN (3,50), ECoL (3,58), PyMFE (3,90), D2V (4,33), Our-Init (4,55) e Random (4,70). Our-Full, TabPFN, ECoL e PyMFE não diferiram significativamente entre si, mas apresentaram desempenho significativamente superior a Our-Init e Random.

Tabela 2. Performance em nível base.

Representação	F1	SERA	Nº de vitórias F1 / SERA (de 215)
TabPFN	0,278	4,844	97 / 56
ECoL	0,274	4,207	82 / 60
Our-Full	0,273	3,902	102 / 60
PyMFE	0,274	4,756	80 / 58
Random	0,249	6,560	48 / 51
D2V	0,247	5,014	71 / 43
Our-Init	0,232	5,539	58 / 58
Oráculo	0,346	1,885	—

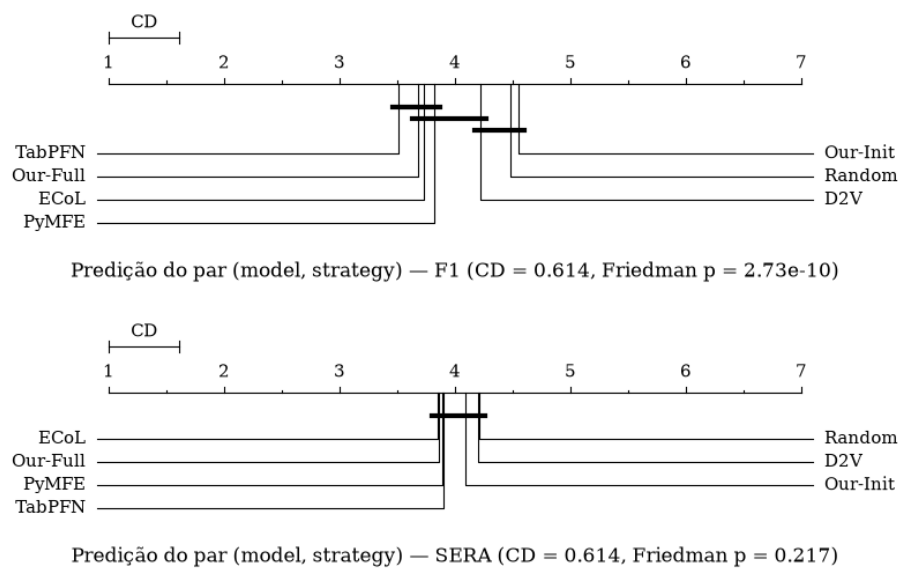


Figura 1. Diagramas de diferença crítica para F1 (em cima) e SERA (em baixo).

Para SERA, o teste de Friedman não rejeitou a hipótese de igualdade ($\chi^2 = 8,37$, $p = 0,213$), e nenhuma diferença entre os rankings médios excedeu a CD , indicando ausência de diferenças estatisticamente significativas entre as representações.

A distribuição dos datasets por nível de desbalanceamento, segundo os limiares de Avelino et al. (2025), foi de 49 datasets no nível baixo, 83 no nível médio e 83 no nível alto. A Figura 2 detalha a quantidade de instâncias em cada nível de desbalanceamento para cada representação.

No nível Médio, o Our-Full apresenta o desempenho mais consistente: lidera tanto em F1 (40 vitórias, contra 38 do TabPFN) quanto em SERA (31, com ampla margem sobre as 23 do TabPFN). No nível Baixo, o cenário se inverte a favor dos meta-features clássicos. Isso sugere que, em datasets mais simples, medidas tradicionais de complexidade já capturam bem as características relevantes. No nível Alto, representações aprendidas como TabPFN (42) e D2V (36) ganham força em F1. Em SERA, porém, o baseline aleatório lidera (26), o que indica que nesse nível as diferenças entre representações são pequenas, os empates são frequentes ou a métrica satura, e a contagem de vitórias perde poder discriminativo.

A comparação entre Our-Full e Our-Init mostra o efeito do treinamento completo. Em F1, o ganho é expressivo e generalizado pois o total de vitórias sobe de 58 para 102, com aumento em

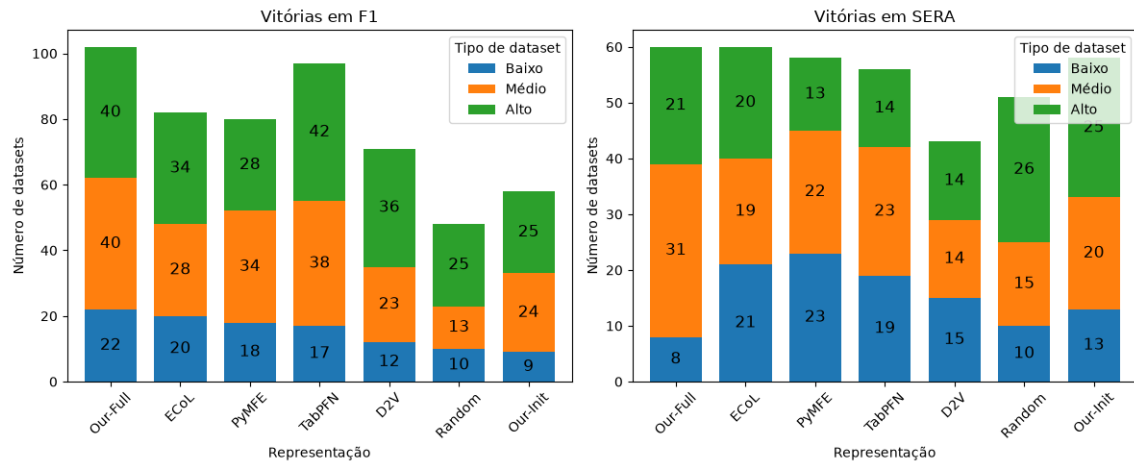


Figura 2. Distribuição das vitórias por representação, nível do dataset e métrica de avaliação.

todos os níveis. Em SERA, o total praticamente não muda (58 para 60), mas a distribuição sim. O modelo treinado ganha no nível Médio (de 20 para 31) e perde nos níveis Baixo (de 13 para 8) e Alto (de 25 para 21). Assim, sob SERA, o treinamento parece especializar a representação em determinado perfil de dataset, em vez de melhorá-la de forma uniforme.

4.2. Seleção de Meta-Features

Nesta subseção, o objetivo foi avaliar se as meta-features de cada representação contêm redundância estatística e informação não discriminativa para a meta-task. Para isso, aplicamos uma seleção em três etapas:

1. **Matriz de correlação.** Para cada representação, calculou-se a matriz de correlação de Spearman entre as meta-features numéricas. Spearman foi escolhido por capturar relações monotônicas, sendo mais robusto do que Pearson a escalas e distribuições heterogêneas entre meta-features.
2. **Agrupamento por redundância.** Com a matriz de correlação, aplicou-se *clustering* hierárquico com distância $1 - |\rho_{\text{Spearman}}|$. Cortando a árvore no limiar correspondente a uma correlação maior ou igual a 0,9, meta-features no mesmo grupo foram tratadas como redundantes, mantendo-se apenas a de maior variância por grupo como representante. Esta etapa removeu exclusivamente por colinearidade, sem considerar poder preditivo.
3. **Recursive Feature Elimination with Cross-Validation.** Então, neste subconjunto de meta-features aplicou-se RFECV (Recursive Feature Elimination with Cross-Validation) com RandomForest como estimador, validação cruzada estratificada e F1 macro como métrica de otimização. O procedimento foi executado independentemente para model e para strategy, pois são meta-tasks distintas e podem depender de subconjuntos diferentes de features. Esta etapa removeu meta-features por relevância preditiva, não por colinearidade.

Foi observado que as representações do TabPFN e Random não apresentaram grupos redundantes. Os embeddings do TabPFN já são suficientemente descorrelacionados entre dimensões, e as meta-features do Random, por serem aleatórias e independentes, não possuem correlação.

O pipeline completo foi então recalculado usando apenas as meta-features selecionadas, e os resultados foram comparados aos obtidos com o conjunto completo original via teste de Wilcoxon pareado ($\alpha = 0,05$). Esses resultados são mostrados na Tabela 4.2. Em nível base, nenhuma

Representação	model		strategy	
	acc	F1-macro	acc	F1-macro
Our-Full	0,679 (-0,000)	0,629 (+0,017)	0,372 (+0,014)	0,264 (+0,020)
TabPFN	0,474 (-0,005)	0,383 (+0,004)	0,488 (+0,037)	0,202 (+0,011)
ECoL	0,465 (+0,005)	0,372 (+0,011)	0,381 (-0,005)	0,158 (+0,004)
PyMFE	0,437 (+0,004)	0,371 (+0,017)	0,381 (+0,046)	0,207 (+0,030)
D2V	0,316 (-0,061)	0,250 (-0,060)	0,353 (+0,065)	0,195 (+0,037)
Our-Init	0,307 (-0,019)	0,265 (-0,004)	0,302 (+0,004)	0,162 (-0,006)
Random	0,288 (-0,000)	0,148 (-0,000)	0,349 (+0,023)	0,161 (+0,062)

Tabela 3. Acurácia e F1-macro na predição dos meta-alvos (model e strategy) após a remoção de meta-features. Verde indica aumento e vermelho indica redução em relação aos resultados anteriores.

das comparações apresentou diferença estatisticamente significativa (todos $p > 0,08$).

A redução de dimensionalidade obtida por agrupamento de redundância seguido de eliminação reversa foi expressiva e não causou perda de desempenho estatisticamente detectável em nenhuma métrica de nível base, nem na maioria das métricas de nível meta. Assim, observa-se que os conjuntos originais de meta-features continham redundância substancial e que a seleção aplicada preserva a capacidade preditiva do pipeline com um número muito menor de variáveis.

5. Conclusões

As meta-features clássicas, concebidas para classificação balanceada, têm aplicabilidade limitada em regressão desbalanceada, e que representações aprendidas genéricas, como o Data-set2Vec, também não capturam bem esse problema. O Our-Full obteve o melhor desempenho no nível meta, graças ao treinamento orientado ao meta-alvo. No nível base, porém, essa vantagem não se sustentou: em F1 não diferiu de TabPFN, ECoL e PyMFE, e em SERA nenhuma representação se distinguiu. Assim, não há evidência de que representações aprendidas ou extraídas sejam superiores nos exemplos raros, nem de que exista uma representação única ideal para todos os níveis de desbalanceamento. Por fim, foi apresentado que a seleção de meta-features reduziu a dimensionalidade sem perda significativa de desempenho.

Referências

aaaa aaaa.

Avelino et al. 2025 Avelino, J. G., Cavalcanti, G. D. C., and Cruz, R. M. O. (2025). Imbalanced regression pipeline recommendation. *Machine Learning*, 114:146.

Moniz et al. 2017 Moniz, N., Branco, P., and Torgo, L. (2017). Evaluation of ensemble methods in imbalanced regression tasks. In *Proceedings of the First International Workshop on Learning with Imbalanced Domains: Theory and Applications*, volume 74 of *Proceedings of Machine Learning Research*, pages 129–140. PMLR.

Vanschoren et al. 2013 Vanschoren, J., van Rijn, J. N., Bischl, B., and Torgo, L. (2013). Openml: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60.

Apêndices

A. Meta-features do PyMFE

A.1. Família *General* (simples)

Meta-característica	Definição	O que captura
nr_inst	Número de instâncias n do dataset	Escala/tamanho da amostra
nr_attr	Número de atributos preditivos m	Dimensionalidade do espaço de entrada
nr_num / nr_cat / nr_bin	Contagem de atributos numéricos / categóricos / binários	Composição de tipos de dado
nr_class	Número de valores distintos do alvo tratados como “classes” pelo PyMFE	Granularidade nominal do alvo (ver ressalva abaixo)
attr_to_inst / inst_to_attr	m/n e seu inverso n/m	Razão dimensionalidade/amostra (risco de <i>curse of dimensionality</i>)
cat_to_num	Razão nº atributos categóricos / numéricos	Heterogeneidade de tipos
freq_class (mean/sd)	Proporção de exemplos por “classe” (valor distinto do alvo), resumida por média/desvio-padrão	Uniformidade da distribuição das categorias do alvo

A.2. Família *Statistical*

Meta-característica	Definição	O que captura
mean, median, min, max, range	Estatísticas univariadas por atributo, resumidas (mean/sd entre atributos)	Escala e centro de cada atributo preditivo
sd, var, iq_range, mad	Desvio-padrão, variância, amplitude interquartil, desvio absoluto mediano por atributo	Dispersão/espalhamento de cada atributo
t_mean	Média aparada (20%) por atributo	Centro robusto a <i>outliers</i>
cov	Elementos da matriz de covariância entre pares de atributos	Relação linear par-a-par entre atributos (X-X)
eigenvalues	Autovalores da matriz de covariância dos atributos	Estrutura de variância multivariada (proporcional às componentes principais)
gravity	Distância entre centróides das “classes” do alvo discretizado	Separação entre grupos do alvo no espaço de atributos
sparsity	Razão (nº valores distintos)/(nº instâncias) por atributo	Esparsidade/granularidade de cada atributo

Meta-característica	Definição	O que captura
<code>nr_cor_attr</code>	Nº/proporção de pares de atributos com $ \text{correlação} $ acima de 0,5	Redundância entre preditores
<code>nr_norm</code>	Nº de atributos que não rejeitam normalidade (teste de Shapiro-Wilk)	Aderência a distribuição gaussiana
<code>nr_outliers</code>	Nº de atributos com valores além de $1,5 \times \text{IQR}$ dos quartis	Presença de <i>outliers</i> nos atributos (X)
<code>nr_disc</code>	Nº de pares atributo–alvo com poder discriminante significativo (teste F, análise discriminante)	Capacidade discriminativa de cada atributo em relação ao alvo discretizado

A.3. Família *Information-theoretic*

Meta-característica	Definição	O que captura
<code>attr_ent</code>	Entropia de Shannon de cada atributo discretizado, resumida	Incerteza/dispersão informacional de cada atributo
<code>class_ent</code>	Entropia de Shannon da distribuição do alvo discretizado	Incerteza da variável-alvo (quão “espalhada” é a distribuição de classes)
<code>joint_ent</code>	Entropia conjunta atributo–alvo, resumida	Incerteza conjunta (X,y)
<code>mut_inf</code>	Informação mútua entre cada atributo e o alvo, resumida	Dependência estatística não-linear atributo $\rightarrow$ alvo
<code>attr_conc</code>	Coefficiente de concentração de Goodman–Kruskal (τ) entre pares de atributos	Associação/concentração entre atributos categóricos
<code>class_conc</code>	Coefficiente de concentração entre atributo e alvo	Associação atributo $\rightarrow$ alvo (via tabela de contingência)
<code>eq_num_attr</code>	<code>class_ent</code> / média de <code>mut_inf</code>	Nº “equivalente” de atributos necessários para explicar o alvo
<code>ns_ratio</code>	$(\text{média}(\text{attr_ent}) - \text{média}(\text{mut_inf})) / \text{média}(\text{mut_inf})$	Razão ruído/sinal: informação irrelevante vs. informação preditiva

B. Meta-features do ECoL

B.1. Família *Complexity*

Meta-característica	Definição	O que captura
T2 — dimensionalidade média	m/n (nº atributos / nº instâncias)	Razão dimensionalidade/amostra (equivalente conceitual ao <code>attr_to_inst</code> do PyMFE, mas na família de complexidade)
T3 — dimensionalidade intrínseca	Nº de componentes principais (PCA) necessários para explicar 95% da variância, dividido por n	Redundância/compressibilidade dos atributos em relação ao tamanho da amostra
T4 — razão de dimensionalidade	(Nº de componentes PCA a 95%) / m	Fração da dimensionalidade original que é de fato “necessária” (atributos redundantes vs. informativos)
L1 — erro absoluto da regressão linear	Erro médio absoluto (MAE) de um modelo de regressão linear múltiplo ajustado a todo o dataset	Quão distante o alvo está de uma relação linear com os atributos
L2 — erro quadrático da regressão linear	Erro quadrático médio (MSE), ou seu complemento normalizado, do mesmo ajuste linear	Idem a L1, com penalização quadrática (mais sensível a poucos erros grandes)
L3 — não-linearidade da regressão linear	Erro do modelo linear ajustado, avaliado em pontos sintéticos gerados por interpolação entre pares de instâncias de treino	Quão bem uma reta se generaliza para <i>regiões</i> entre os exemplos observados (não só nos pontos originais)
C2 — correlação atributo–alvo (complemento)	1 – correlação (absoluta, ex. Spearman) média entre atributos e alvo	Ausência de relação monotônica direta entre qualquer atributo isolado e o alvo
C3 — eficiência individual de atributo	Para o atributo mais correlacionado ao alvo, proporção de instâncias que precisam ser removidas para a correlação atingir um limiar alto	Quantos exemplos “atrapalham” a melhor relação univariada — sensível a poucos pontos discordantes
C4 — eficiência coletiva de atributos	Extensão iterativa de C3 usando combinações de atributos até atingir o limiar de correlação	Quantos exemplos permanecem inexplicados mesmo usando vários atributos em conjunto
S1 — suavidade de saída	Diferença média $ y_i - y_j $ entre pares conectados pela Árvore Geradora Mínima (MST) no espaço de atributos, normalizada	Quão abruptamente o alvo muda entre vizinhos próximos (rugosidade da função-alvo)
S2 — suavidade de entrada	Distância média entre pares conectados pela MST, no espaço de atributos	Irregularidade/densidade da distribuição dos pontos de entrada
S3 — erro do 1-NN	Erro (MAE, <i>leave-one-out</i>) de um regressor por vizinho mais próximo	Previsibilidade local a partir de vizinhança imediata

Meta-característica	Definição	O que captura
S4 — não-linearidade do 1-NN	Erro do 1-NN avaliado em pontos interpolados sinteticamente (análogo a L3, mas para vizinhança)	Se a estrutura local de vizinhança generaliza para regiões intermediárias